

Object-Oriented Programming

@author Yangxin Wu

Last edited: 20251129

- [面向对象的三大特点](#)
 - [封装](#)
 - [继承](#)
 - [多态](#)
- [面向对象的七个原则](#)
 - [单一职责原则](#)
 - [开闭原则](#)
 - [里氏代换原则](#)
 - [依赖倒转原则](#)
 - [合成复用原则](#)
 - [接口隔离原则](#)
 - [迪米特法则](#)
- [UML 类图](#)
 - [类和接口的表示](#)
 - [类图中关系的表示](#)
- [设计模式](#)
 - [单例模式](#)
 - [简单工厂模式](#)
 - [工厂模式](#)
 - [抽象工厂模式](#)
 - [迭代器模式](#)
 - [策略模式](#)
 - [数据访问对象模式](#)
 - [观察者模式](#)
 - [模板方法模式](#)

面向对象的三大特点

所谓面向对象，即：

- 任何系统都是由能够完成一组相关任务的对象构成。
- 如果对象依赖于一个不属于它负责的任务，那么就需要访问负责此任务的另一个对象。
- 一个对象不能直接操作另一个对象内部的数据，它也不能使其它对象直接访问自己的数据。
- 所有的「交流」都是通过「方法」的调用完成。

封装

- 隐藏对象的属性和实现细节，仅对外公式访问方法。
- 增加安全性和简化编程。

继承

- 子类继承父类的特征和行为。
- 实现代码的复用。

多态

- 同一个行为具有不同表现性态的能力（「一个接口，多个方法」）。
- 提高了程序的扩展性和可维护性。

面向对象的七个原则

在设计面向对象的程序时，或者说，安排程序中的类和对象时，我们需要遵循七个原则。

单一职责原则

就一个类而言，应该仅有一个能引起它变化的原因。

- 不同变化 $\Rightarrow$ 不相干的职责 $\Rightarrow$ 难以复用。
- 如果一个类有多于一个的职责，考虑职责的分离。

开闭原则

一个软件实体应「对扩展开放，对修改关闭」：

- 通过抽象类和接口，规定了具体类的特征，实现「对修改关闭」。
- 利用重载改变具体类，可以改变系统的行为，实现「对扩展开放」。

例如，有下面的这些「水果」类：

```
class Fruit { ... }           // 父类，水果
class Apple extends Fruit { ... } // 子类，苹果
class Orange extends Fruit { ... } // 子类，橙子
class Banana extends Fruit { ... } // 子类，香蕉
```

如果想要引入一款新的电脑，不需要动上述任何代码（所谓的「闭」），只要增加（所谓的「开」）

```
class Passionfruit extends Fruit { ... } // 子类，百香果
```

里氏代换原则

使用父类的地方一定可以换成子类，反之不然。

```
Fruit a = new Orange();    // 没问题
Orange b = new Orange();   // 没问题
Orange c = new Fruit();    // 大问题
```

依赖倒转原则

高层模块不要依赖低层模块，而应该使用接口。

接上面水果的例子，若定义用户的「吃水果」函数为

```
class User {
    public void eatFruit(Orange obj) { ... }
}
```

那么用户就和 `Orange` 这一个类绑定了，后续更换变得麻烦。这就是「高层模块依赖低层模块」。

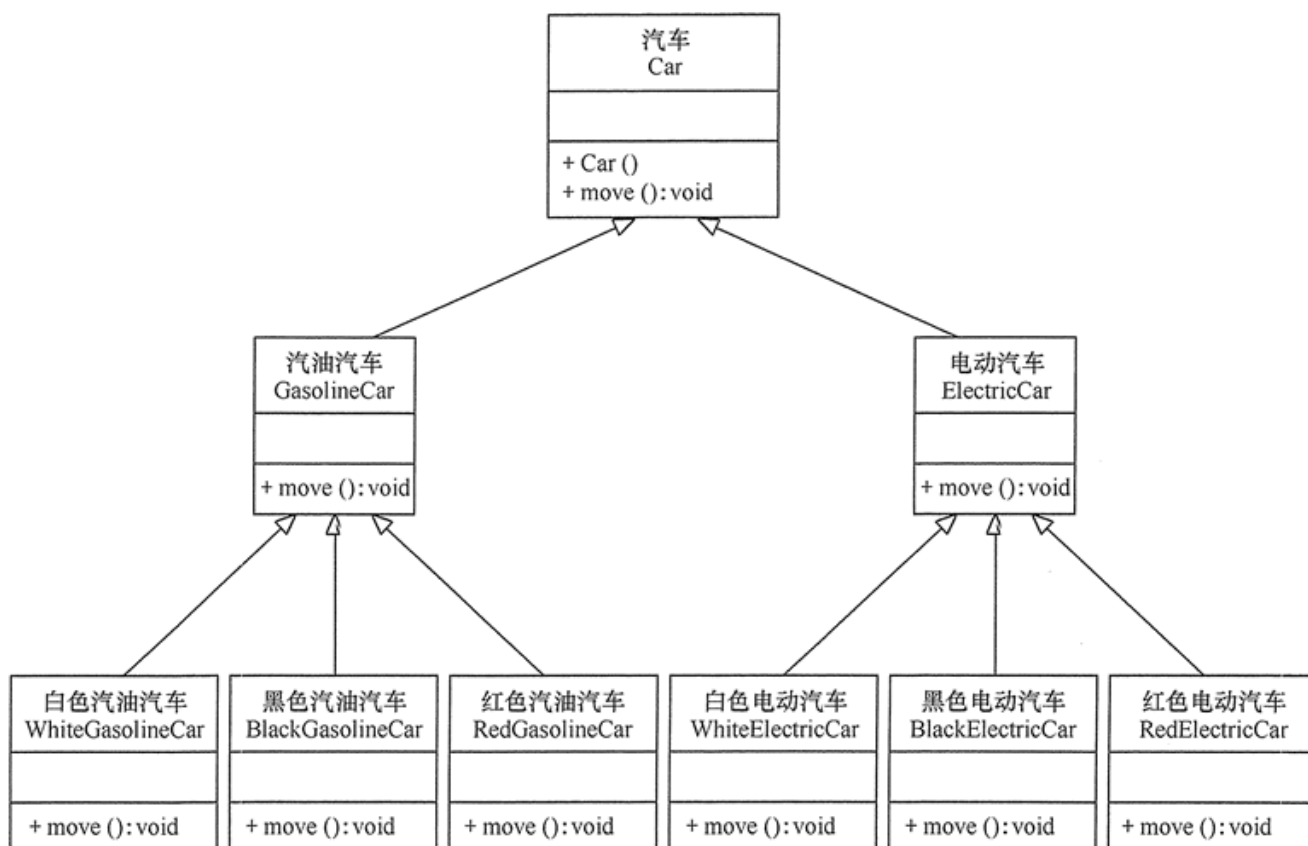
更好的写法应是

```
class User {
    public void eatFruit(Fruit obj) { ... }
}
```

合成复用原则

尽量使用对象组合，而不是继承来达到复用的目的。即：在一个新的对象里面 **使用**（聚合关系）一些已有的对象，使之成为新对象的一部分。

例如：汽车 `Car` 按动力划分可以分为油车、电车，按颜色划分可以分成黑车、白车、红车等。如果用继承的方法实现它们，组合种类会非常多：



反过来，如果将「发动机」 Engine、「颜色」 Color 作为 Car 的内部字段，则能大大地降低代码的复杂度。这就是合成复用。

接口隔离原则

客户端不应该依赖它不需要的接口。换句话说，使用多个专门的接口比使用单一的总接口要好。一旦一个接口太大，则需要将它分割成一些更细小的接口。

例如，定义下面的接口「日常生活」

```
interface DailyLife {
    public void sing();
    public void move();
    public void think();
    public void mate();
    ...
}
```

然后让 Person 类去实现它。这样造成的问题是，智能手机也能 sing 和 think，但不能 move 和 mate。那么，当我们为「智能手机」编写相关的类时，就得重新做一个接口「智能手机的日常生活」。解决方案是将这个大接口拆分：

```
interface Singable {
    public void sing();
}

interface Moveable {
    public void move();
}
```

```
}

interface Thinkable {
    public void think()
}

interface Mateable {
    public void mate();
}
```

因为 Java 中一个类可以实现多个接口，所以

```
class Person implements Singable, Moveable, Thinkable, Mateable { ... }
class Phone implements Singable, Thinkable { ... }
```

迪米特法则

一个软件实体应当尽可能少的与其他实体发生相互作用。每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

UML 类图

统一建模语言（Unified Modeling Language, [UML](#)）是一种设计软件蓝图的可视化建模语言。

使用 UML 绘制的类与类之间的关系图称为类图。

类和接口的表示

- + 表示 public ;
- - 表示 private ;
- # 表示 protected ;
- 不带符号表示 friendly。

类图中关系的表示

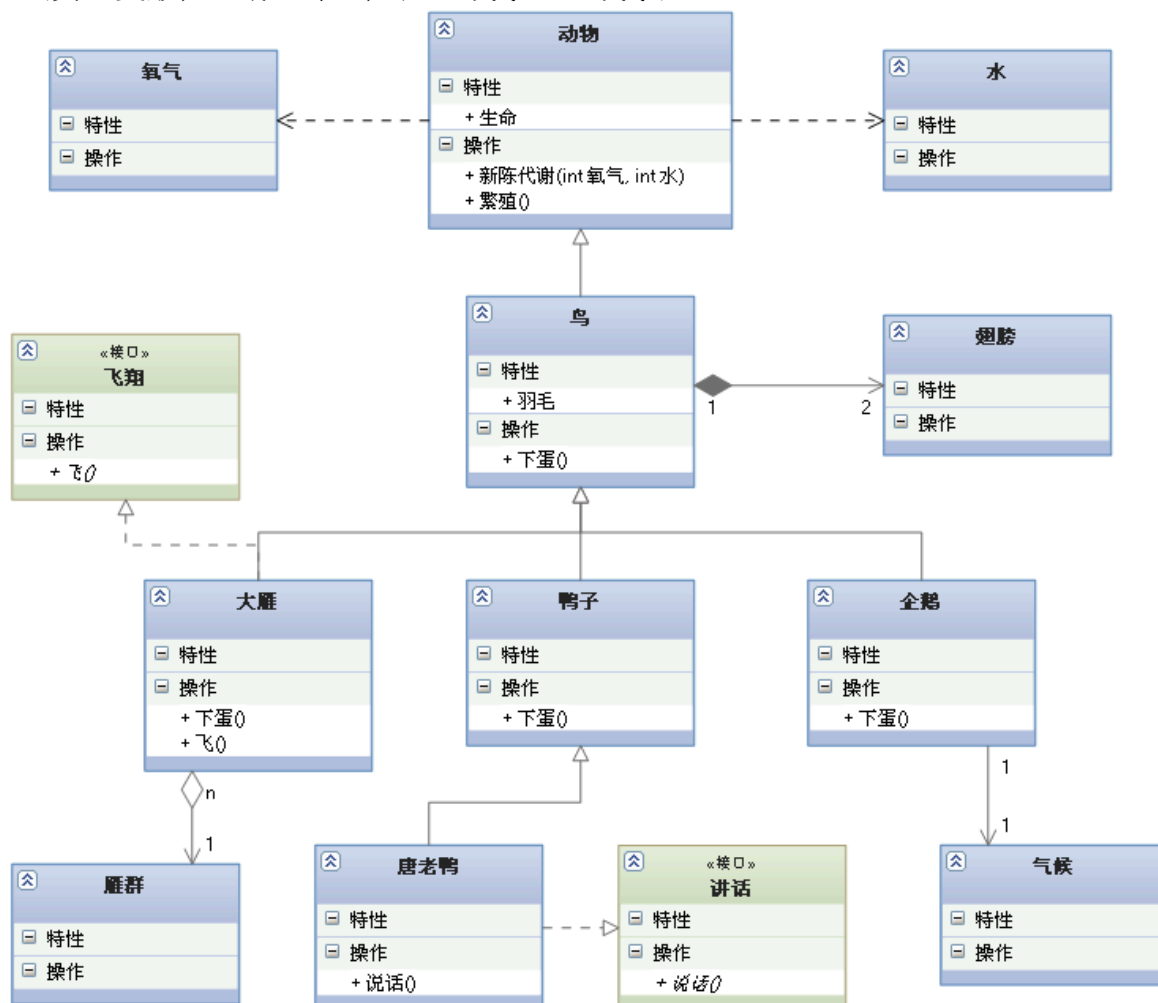
参见 <http://www.uml.org.cn/oobject/201609062.asp>

- 泛化（三角箭头实线，即继承 extends）
- 实现（三角箭头虚线，即实现 implements）
- 关联（普通箭头实线，A 与 B 有关系，如妻子与丈夫）

- 依赖（普通箭头虚线，A 依赖 B，如动物和氧气）

依赖关系的类型	关键字或构造型	描述
抽象	«abstraction»、«derive»、«refine» 或 «trace»	使在不同抽象级别或者从不同视点来表示同一概念的两个模型元素或多组模型元素相关
绑定	«bind»	将模板自变量连接至模板参数以根据模板来创建模型元素
实现	«realize»	指示客户模型元素是供应者模型元素的实现，而供应者模型元素是规范
替换	«substitute»	指示客户模型元素替代供应者；客户模型元素必须遵照供应者模型元素建立的合同或接口
使用	«use»、«call»、«create»、«instantiate» 或 «send»	指示一个模型元素需要另一个模型元素才能完全实现或执行操作

- 聚合（空心菱形，A 有一个 B，但 A 没了 B 也可以有）
- 组合（实心菱形，A 有一个 B，但 A 没了 B 也没了）



设计模式

一套被反复使用、经过分类编目的代码设计经验的总结。

设计模式根据「模式是用来完成什么工作的」来分，可以分为「创建型模式」「结构型模式」和「行为型模式」三种；根据「模式应用于类还是对象」，可以分为「类模式」和「对象模式」两种。

单例模式

使用静态变量，保证一个类仅有一个实例，并提供一个访问它的全局访问点。

实现对唯一实例的受控访问，减少了资源的开销，并能全局访问，避免对共享资源的多重占用。

单例模式允许可变数目的实例。

- 饿汉式

在类加载的时候立刻会实例化，后续使用就只会出现一份实例。

```
public class Singleton {
    // 在类初始化时立即加载对象
    private static Singleton instance = new Singleton();

    // 将构造方法声明为私有，使外界无法利用 new 创建此类实例
    private Singleton() {}

    private static Singleton GetInstance() {
        return instance;
    }
}
```

- 懒汉式

不在类加载的时候直接实例化，而是调用指定实例方法的时候再进行实例化，这样就能保证不想使用的时候也不会实例化。

```
public class Singleton {
    private static Singleton instance;

    // 将构造方法声明为私有，使外界无法利用 new 创建此类实例
    private Singleton() {}

    // 利用公有的静态方法 getInstance() 来实例化并返回唯一实例
    public static Singleton GetInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- 同步锁

在懒汉式的基础上，如果多个线程同时访问 Singleton 类，调用 getInstance() 方法，则仍有可能出现创建多个实例的方法。

在操作系统进程锁的启发下，我们通过增加 synchronized 关键字到 getInstance() 方法中，迫使每个线程再进入这个方法之前，要先等候别的线程离开该方法。

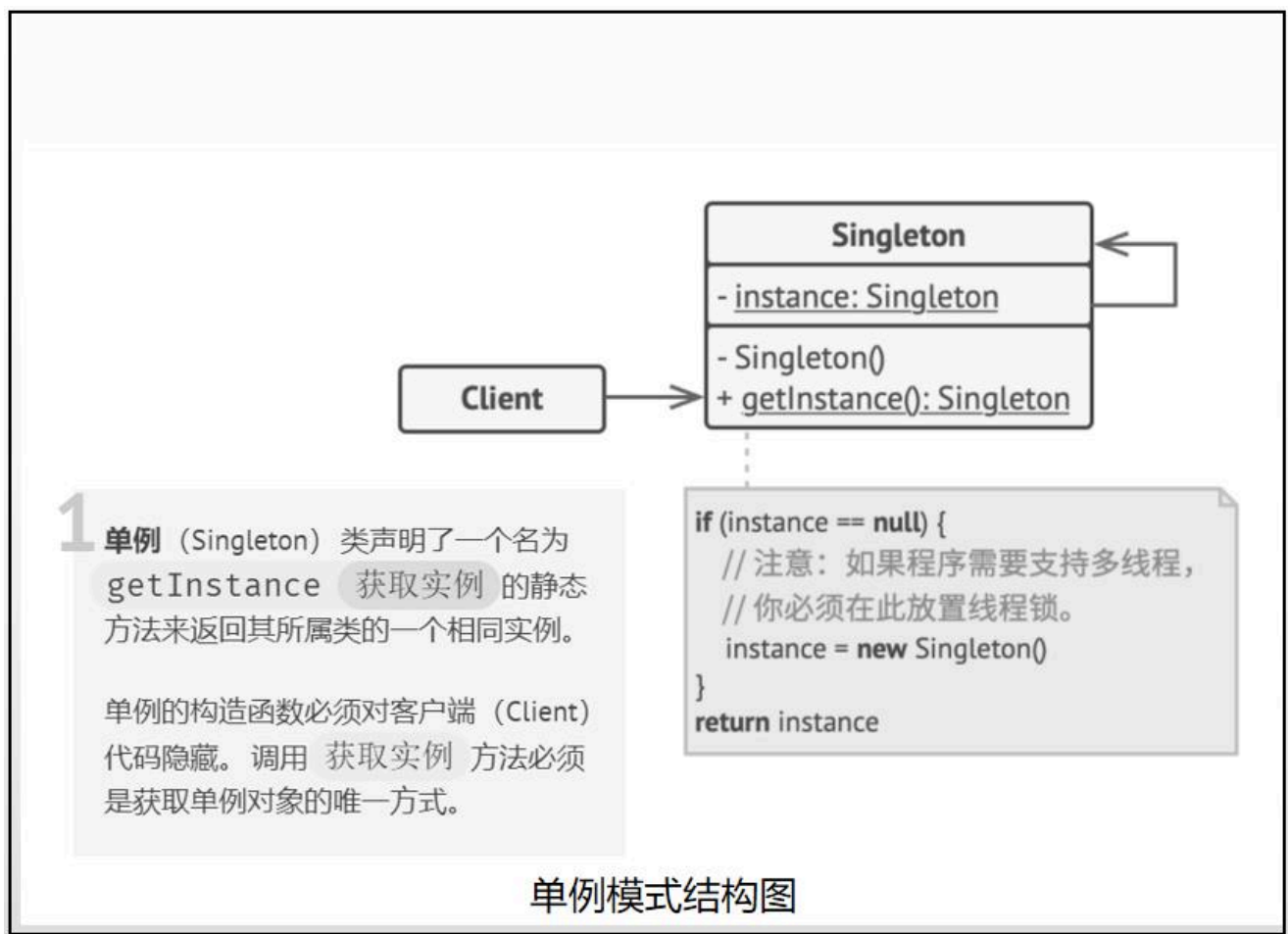
```

public class Singleton {
    private static Singleton instance;

    // 将构造方法声明为私有，使外界无法利用 new 创建此类实例
    private Singleton() {}

    // 利用公有的静态方法 getInstance() 来实例化并返回唯一实例
    public static synchronized Singleton GetInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```



简单工厂模式

简单工厂模式提供专门的工厂类用于创建对象，实现了对象创建和使用的职责分离，客户端不需知道所创建的具体产品类的类名以及创建过程，只需知道具体产品类所对应的参数即可，通过引入配置文件，可以在不修改任何客户端代码的情况下更换和增加新的具体产品类。

```

/* 产品 */
abstract class BMW {
    public BMW() {}
}

```



```

}

public class BMW320 extends BMW {
    public BMW320() {
        System.out.println("制造 -> BMW320");
    }
}

public class BMW523 extends BMW {
    public BMW523() {
        System.out.println("制造 -> BMW523");
    }
}

/* 工厂 */
public class Factory {
    public BMW createBMW(int type) {
        switch (type) {
            case 320: return new BMW320();
            case 523: return new BMW523();
            default: break;
        }
        return null;
    }
}

/* 用户 */
public class Customer {
    public static void main(String[] args) {
        Factory factory = new Factory();
        BMW bmw320 = factory.createBMW(320);
        BMW bmw523 = factory.createBMW(523);
    }
}

```

缺点在于：

- 不符合“开闭原则”，每次添加新产品就需要修改工厂类。在产品类型较多时，有可能造成工厂逻辑过于复杂，不利于系统的扩展维护；
- 工厂类集中了所有产品创建逻辑，一旦不能正常工作，整个系统都要受到影响。

工厂模式

工厂方法模式将工厂抽象化，并定义一个创建对象的接口。每增加新产品，就增加该产品以及对应的具体实现工厂类，由具体工厂类决定要实例化的产品是哪个，将对象的创建与实例化延迟到子类

```

/* 产品 */
...

/* 工厂 */
interface FactoryBMW {
    BMW createBMW();
}

public class FactoryBMW320 implements FactoryBMW {
    @Override
    public BMW320 createBMW() {
        return new BMW320();
    }
}

public class FactoryBMW523 implements FactoryBMW {
    @Override
    public BMW523 createBMW() {
        return new BMW523();
    }
}

/* 用户 */
public class Customer {
    public static void main(String[] args) {
        FactoryBMW320 factoryBMW320 = new FactoryBMW320();
        BMW bmw320 = factoryBMW320.createBMW();

        FactoryBMW523 factoryBMW523 = new FactoryBMW523();
        BMW bmw523 = factoryBMW523.createBMW();
    }
}

```

缺点在于：

每增加一个产品都需要增加一个具体产品类和实现工厂类，使得系统中类的个数成倍增加，在一定程度上增加了系统的复杂度，同时也增加了系统具体类的依赖。

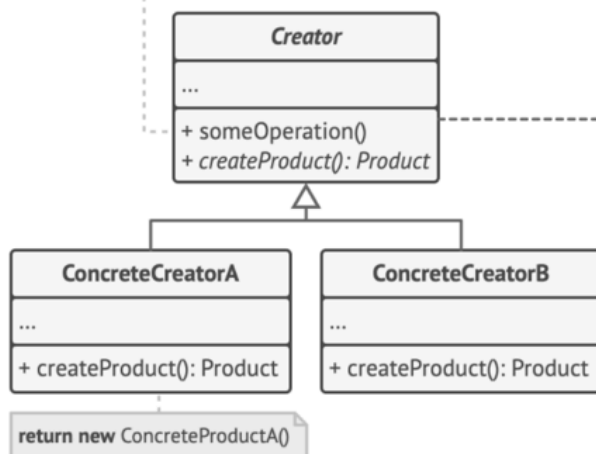
3

创建者 (Creator) 类声明返回产品对象的工厂方法。该方法的返回对象类型必须与产品接口相匹配。

你可以将工厂方法声明为抽象方法，强制要求每个子类以不同方式实现该方法。或者，你也可以在基础工厂方法中返回默认产品类型。

注意，尽管它的名字是创建者，但它最主要的职责并不是创建产品。一般来说，创建者类包含一些与产品相关的核心业务逻辑。工厂方法将这些逻辑处理从具体产品类中分离出来。打个比方，大型软件开发公司拥有程序员培训部门。但是，这些公司的主要工作还是编写代码，而非生产程序员。

```
Product p = createProduct()
p.doStuff()
```

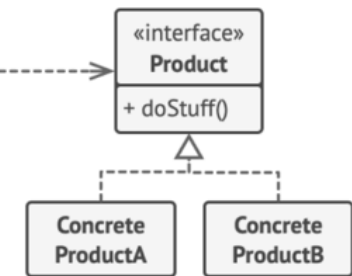


4 具体创建者 (Concrete Creators) 将会重写基础工厂方法，使其返回不同类型的产品。

注意，并不一定每次调用工厂方法都会创建新的实例。工厂方法也可以返回缓存、对象池或其他来源的已有对象。

1

产品 (Product) 将会对接口进行声明。对于所有由创建者及其子类构建的对象，这些接口都是通用的。



2 具体产品 (Concrete Products) 是产品接口的不同实现。

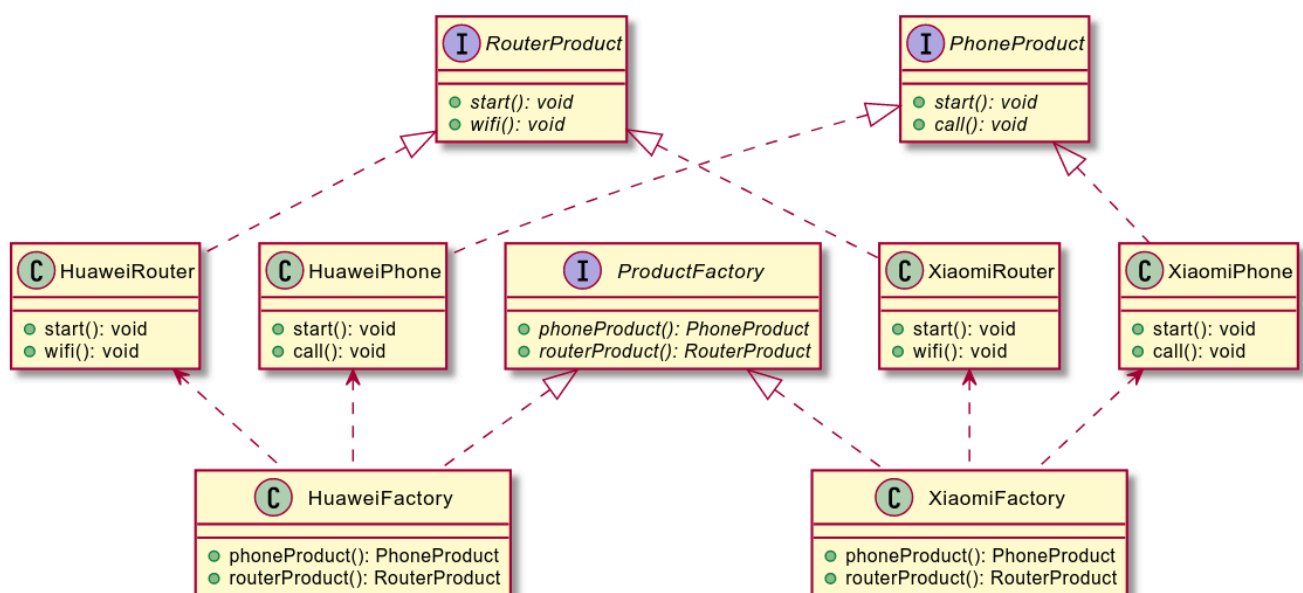
工厂模式结构图

抽象工厂模式

适用于有多个产品族的情景。

围绕一个超级工厂创建其他工厂，该超级工厂又称为其他工厂的工厂。

使用这种模式，添加「产品等级」（在下例中，为增加一个手机品牌）相当简单，只要让工厂实现 `ProductFactory` 接口，让自己的产品分别实现 `PhoneProduct` 和 `RouterProduct` 接口就可以。



缺点在于：

- 要增加产品族（下例中，为增加一个新的产品类型，如「平板电脑」）十分困难。

迭代器模式

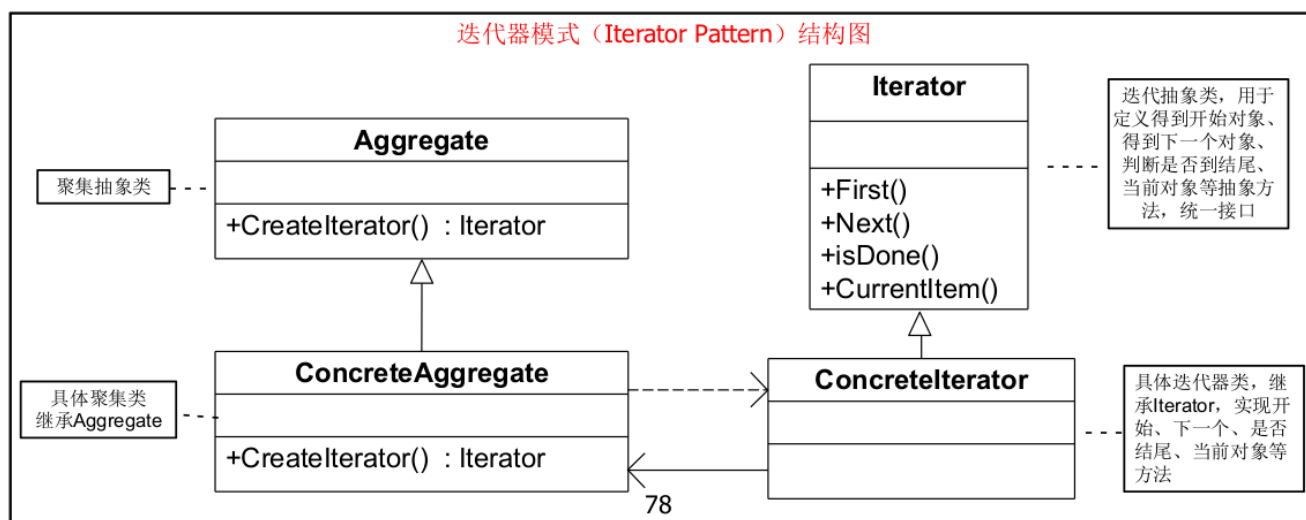
对聚合类（例如 List 和 Set）的一种访问方式。

使用 `iterator()` 方法可以从支持迭代器模式的聚合类中获得迭代器 `Iterator`。

对迭代器有下面的常用方法：

- `next()`：返回迭代器的下一个元素，并且更新迭代器的状态。
- `hasNext()`：检测集合中是否还有元素。
- `remove()`：迭代器返回的元素删除。

迭代器使得对于序列类型数据结构的遍历与被遍历的对象分离，我们无需关心该序列的底层结构。



实例代码如下：

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorTest {
    public static void main(String[] args) {
        ArrayList<String> comps = new ArrayList<String>();

        comps.add("Google");
        comps.add("Amazon");
        comps.add("Apple");
        comps.add("Meta");
        // comps: [Google, Amazon, Apple, Meta]

        // 获取迭代器
        Iterator<String> iter = comps.iterator();

        // 输出集合中的第一个元素
        System.out.println(iter.next());
        // Google

        while (iter.hasNext()) {
```

```
        System.out.println(iter.next());
    }
    // Google
    // Amazon
    // Apple
    // Meta

    while (iter.hasNext()) {
        String str = iter.next();
        if (str.length() < 5) {
            iter.remove();
        }
    }
    // comps: [Google, Amazon, Apple]
}
}
```

缺点：

- 对于比较简单的遍历（数组或者有序列表），使用迭代器方式遍历较为繁琐且效率不高。
- 使用迭代器的方式比较适合那些底层以链表形式实现的集合。

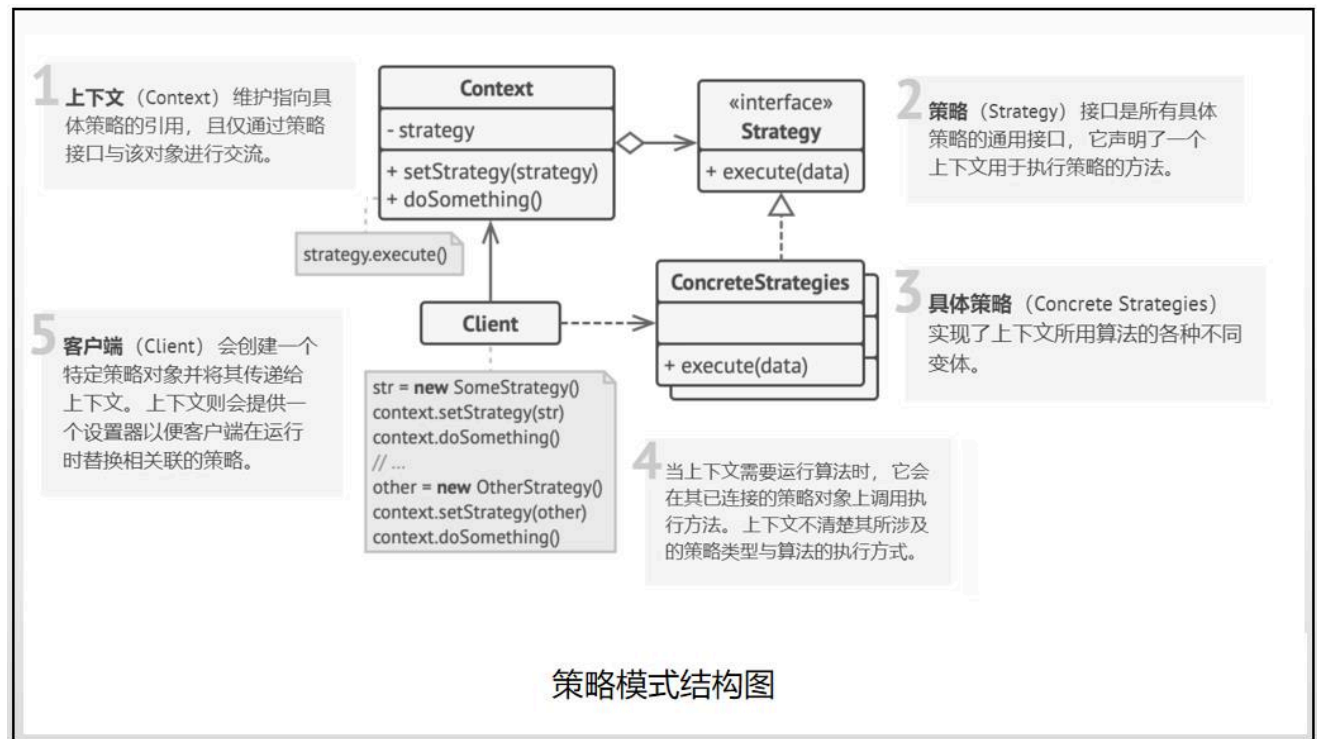
策略模式

策略模式将对象与算法分离，使得在有多种算法相似的情况下，避免多个 `if-else` 语句带来的复杂性和难以维护性。

策略模式涉及到三个角色：

- 抽象策略（**Strategy**）：这是一个抽象角色，通常由一个接口或抽象类实现。此角色给出所有的具体策略类所需的接口。
- 具体策略（**ConcreteStrategy**）：包装了相关的算法或行为。
- 环境（**Context**）：持有一个 **Strategy** 的引用。

使用策略模式的优点：算法可以自由切换，扩展性好；但增加暴露的策略类可能不易维护。



以下是应用策略模式的一个例子：

```
/* 抽象策略 */
public interface MemberStrategy {
    public double calcPrice(double booksPrice);
}

/* 具体策略 1 */
public class PrimaryMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("初级会员没有折扣");
        return booksPrice;
    }
}

/* 具体策略 2 */
public class IntermediateMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("中级会员打九折");
        return booksPrice * 0.9;
    }
}

/* 具体策略 3 */
public class AdvancedMemberStrategy implements MemberStrategy {
    @Override
    public double calcPrice(double booksPrice) {
        System.out.println("高级会员打八折");
    }
}
```

```

        return booksPrice * 0.8;
    }
}

/* 环境 */
public class Network {
    private MemberStrategy strategy;

    /* 构造函数，传入一个具体的策略对象 */
    public Network(MemberStrategy strategy) {
        this.strategy = strategy;
    }

    /* 计算图书的价格 */
    public double quote(double booksPrice) {
        return this.strategy.calcPrice(booksPrice);
    }
}

/* 客户端 */
public class Client {
    public static void main(String[] args) {
        // 选择并创建需要使用的策略对象
        MemberStrategy strategy = new AdvancedMemberStrategy();

        // 创建环境
        Network network = new Network(strategy);

        // 计算价格
        double quote = network.quote(300);

        System.out.println("图书的最终价格为: " + quote);
    }
}

```

策略模式的重心不是如何实现算法，而是如何组织、调用这些算法，从而让程序结构更灵活，具有更好的维护性和扩展性。

缺点：

- 客户端必须知道所有的策略类，并自行决定使用哪一个策略类。这就意味着客户端必须理解这些算法的区别，以便适时选择恰当的算法类。
- 策略模式把每个具体的策略实现都单独封装成为类，若备选策略很多，对象的数目就会很可观。

数据访问对象模式

用于将低级的数据操作从高级的业务逻辑中分离。

DAO 模式的参与者：

- 数据访问对象接口：该接口定义了一个模型对象上要执行的标准操作。
- 数据访问对象实体类：该类实现了上述的接口，负责从数据源获取数据，数据源可以是数据库，也可以是 **xml**，或者是其他的存储机制。
- 模型对象/数值对象：该对象是简单的普通对象，包含了 `get / set` 方法来存储通过使用 **DAO** 类检索到的数据。

实例代码如下：

```
// 创建数值对象
public class Student {
    private String name;
    private int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getRollNo() {
        return rollNo;
    }

    public void setRollNo(String name) {
        this.rollNo = rollNo;
    }
}

// 创建数据访问对象接口
public interface StudentDao {
    List<Student> getAllStudents();

    Student getStudent(int rollNo);

    void updateStudent(Student student);

    void deleteStudent(Student student);
}

// 创建实现了上述接口的实体类
public class StudentDaoImpl implements StudentDao {
```



```

List<Student> students;

public StudentDaoImpl() {
    students = new ArrayList<Student>();
    Student student1 = new Student("Robert", 0);
    Student student2 = new Student("John", 1);
    students.add(student1);
    students.add(student2);
}

@Override
public void deleteStudent(Student student) {
    students.remove(student.getRollNo());
    System.out.println(
        "Student: Roll No " + student.getRollNo() + ", deleted."
    );
}

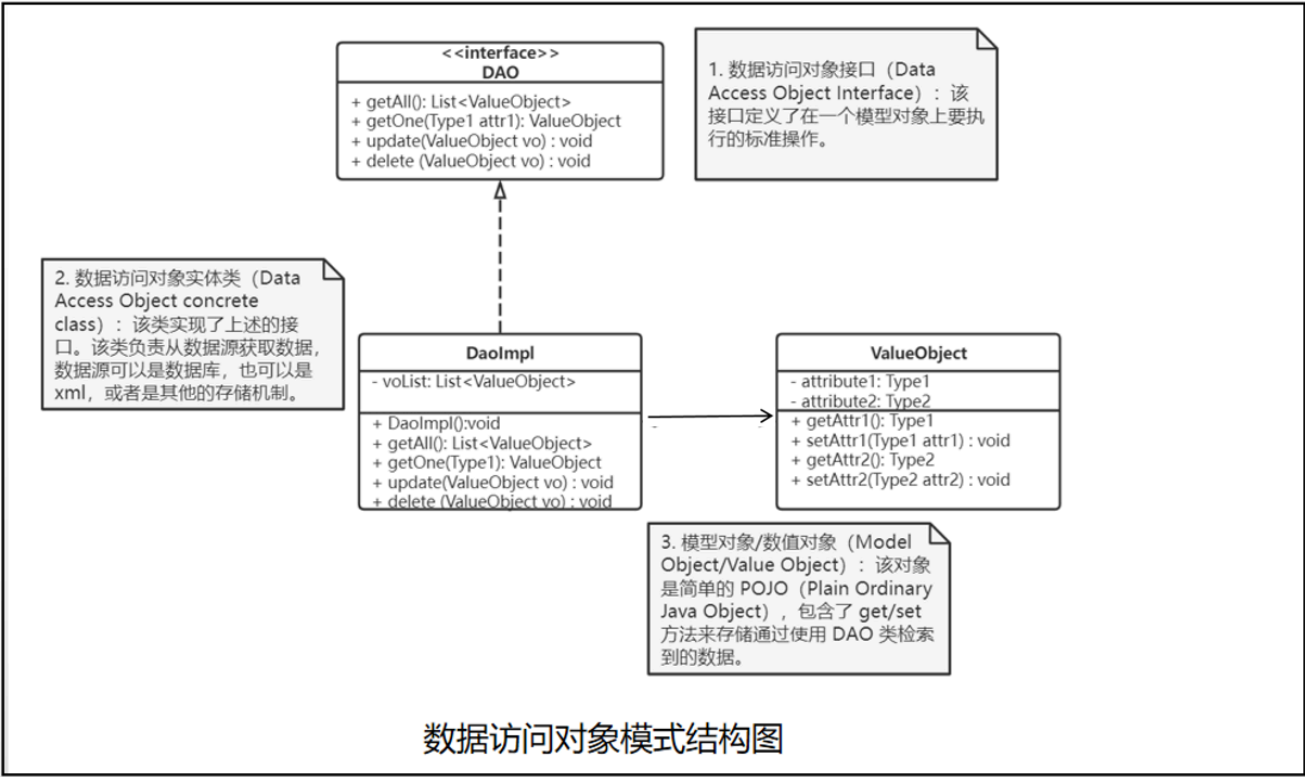
@Override
public List<Student> getAllStudents() {
    return students;
}

@Override
public Student getStudent(int rollNo) {
    return students.get(rollNo);
}

@Override
public void updateStudent(Student student) {
    students.get(student.getRollNo()).setName(student.getName());
    System.out.println(
        "Student: Roll No " + student.getRollNo() + ", updated."
    );
}
}

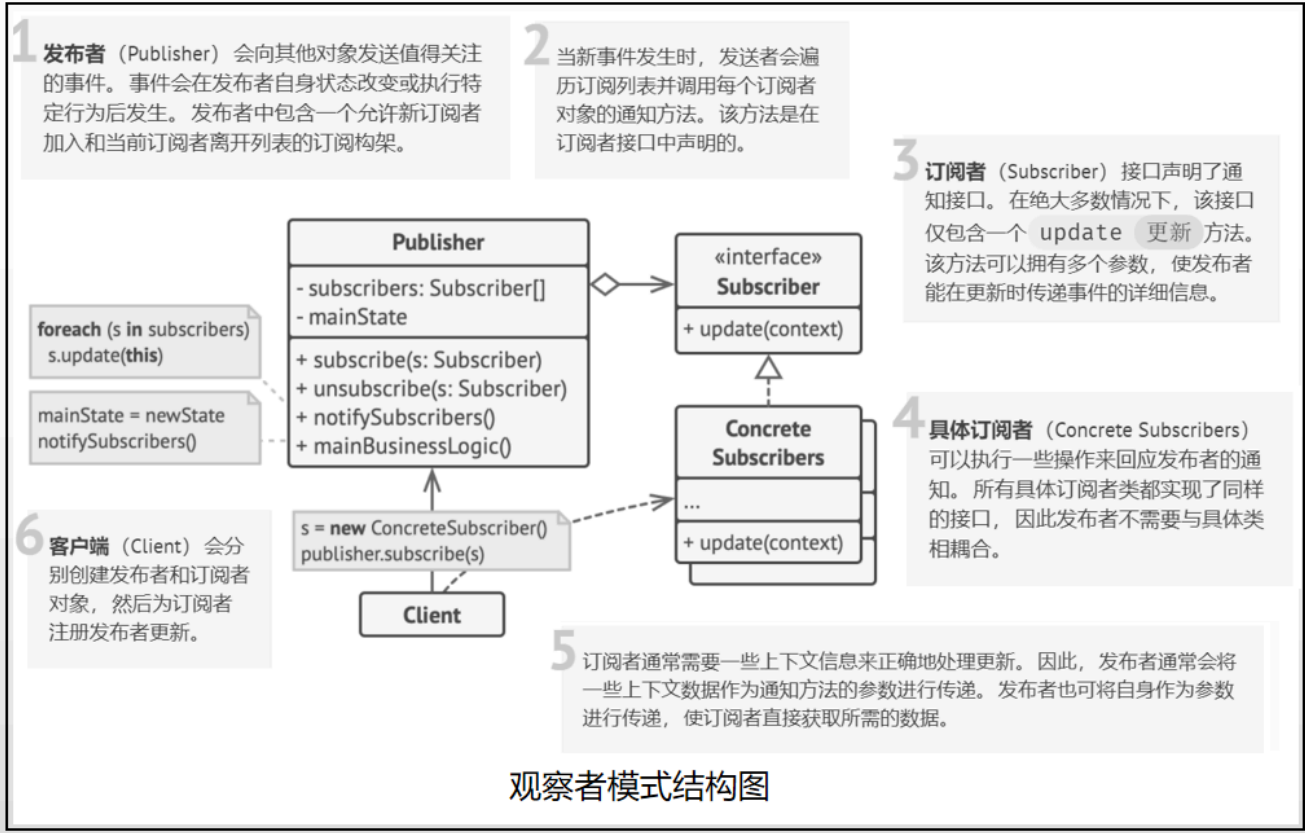
// 演示
// ...

```



观察者模式

定义了对对象之间一种一对多的依赖关系, 让一个对象的改变能够影响其他对象。



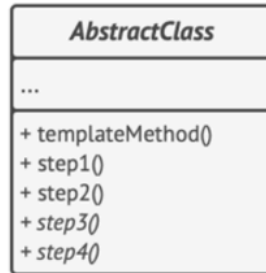
模板方法模式

定义一个操作中的算法骨架, 将一些步骤延迟到子类中, 使得子类可以不改变一个算法的具体结构即可重新定义该算法的特定步骤。

- 钩子操作（可以重新定义）
- 抽象操作（必须重新定义）

1

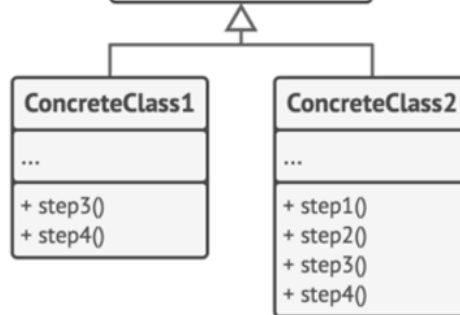
抽象类（AbstractClass）会声明作为算法步骤的方法，以及依次调用它们的实际模板方法。算法步骤可以被声明为 抽象 类型，也可以提供一些默认实现。



```
step1()
if (step2()) {
    step3()
}
else {
    step4()
}
```

2

具体类（ConcreteClass）可以重写所有步骤，但不能重写模板方法自身。



模板模式结构图